

Swinburne University of Technology*School of Science, Computing and Emerging Technologies***ASSIGNMENT COVER SHEET**

Subject Code: COS30008
Subject Title: Data Structures & Patterns
Assignment number and title: 3 – Amortized Analysis & Abstract Data Types
Due date: Sunday, May 24, 2026, 23:59
Lecturer: Dr. Markus Lumpe

Your name: Dimitrios Moros **Your student ID:** 103598757

Marker's comments:

Problem	Marks	Obtained
1	34	
2	112	
Total	146	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener: _____

```
1
2 // COS30008 - Problem Set 3, 2026
3
4 #pragma once
5
6 #include <ostream>
7
8 template<typename K, typename V>
9 class SortablePair
10 {
11 public:
12
13     SortablePair( const K& aFirst = K{}, const V& aSecond = V{} ) noexcept;
14
15     const K& first() const noexcept;
16     const V& second() const noexcept;
17
18     bool operator==( const SortablePair& aOther ) const noexcept;
19
20     bool operator<( const SortablePair& aOther ) const noexcept;
21
22     friend std::ostream& operator<<( std::ostream& aOStream, const      ↗
        SortablePair<K,V>& aPair );
23
24 private:
25
26     K fFirst;
27     V fSecond;
28 };
29
30
```

```
1 #include "SortablePair.h"
2
3 template<typename K, typename V>
4 SortablePair<K, V>::SortablePair( const K& aFirst, const V& aSecond )    ↗
    noexcept
5     : fFirst(aFirst), fSecond(aSecond)
6 {
7
8 }
9
10 template<typename K, typename V>
11 const K& SortablePair<K, V>::first() const noexcept
12 {
13     return fFirst;
14 }
15
16 template<typename K, typename V>
17 const V& SortablePair<K, V>::second() const noexcept
18 {
19     return fSecond;
20 }
21
22 template<typename K, typename V>
23 bool SortablePair<K, V>::operator==( const SortablePair& aOther ) const    ↗
    noexcept
24 {
25     return fFirst == aOther.fFirst && fSecond == aOther.fSecond;
26 }
27
28
29 template<typename K, typename V>
30 bool SortablePair<K, V>::operator<( const SortablePair& aOther ) const    ↗
    noexcept
31 {
32     return fFirst > aOther.fFirst;
33 }
34
35
36
37 template class SortablePair<size_t, int>;
38 std::ostream& operator<<( std::ostream& aOStream, const    ↗
    SortablePair<size_t, int>& aPair )
39 {
40     aOStream << '(' << aPair.fFirst << ',' << aPair.fSecond << ')';
41     return aOStream;
42 }
43
```

```
1
2 // COS30008 - Problem Set 3, 2026
3
4 #pragma once
5
6 #include "SortablePair.h"
7
8 #include <optional>
9 #include <cassert>
10 #include <algorithm>
11
12 template<typename T, typename P>
13 class PriorityQueue
14 {
15 public:
16     using value_type = SortablePair<P, T>;
17
18     PriorityQueue() noexcept;
19
20     ~PriorityQueue() noexcept;
21
22     PriorityQueue( const PriorityQueue& ) = delete;
23     PriorityQueue& operator=( const PriorityQueue& ) = delete;
24
25     size_t count() const noexcept;
26     size_t capacity() const noexcept;
27
28     std::optional<T> top() const noexcept;
29     void push( const T& aValue, const P& aPriority ) noexcept;
30     void pop() noexcept;
31
32 private:
33     value_type* fElements;
34     size_t fHead;
35     size_t fTail;
36     size_t fCapacity;
37
38     void sort() noexcept
39     {
40         std::sort( &fElements[fHead], &fElements[fTail] );
41     }
42
43     void resize( size_t aCapacity );
44     void ensure_capacity();
45     void adjust_capacity();
46 };
47
48
49
```

```
50 template<typename T, typename P>
51 PriorityQueue<T, P>::PriorityQueue() noexcept
52     : fElements(static_cast<value_type*> (::operator new(sizeof
53         (value_type))))),
54     fHead(0),
55     fTail(0),
56     fCapacity(1)
57 {
58 }
59 template<typename T, typename P>
60 PriorityQueue<T, P>::~~PriorityQueue() noexcept
61 {
62     for (size_t i = fHead; i < fTail; i++)
63     {
64         fElements[i].~value_type();
65     }
66     ::operator delete(fElements);
67 }
68
69 template<typename T, typename P>
70 size_t PriorityQueue<T, P>::count() const noexcept
71 {
72     return fTail - fHead;
73 }
74
75 template<typename T, typename P>
76 size_t PriorityQueue<T, P>::capacity() const noexcept
77 {
78     return fCapacity;
79 }
80
81 template<typename T, typename P>
82 std::optional<T> PriorityQueue<T, P>::top() const noexcept
83 {
84     if (count() == 0)
85     {
86         return std::nullopt;
87     }
88     return fElements[fHead].second();
89 }
90
91 template<typename T, typename P>
92 void PriorityQueue<T, P>::resize(size_t aCapacity)
93 {
94     value_type* lNewElements = static_cast<value_type*> (::operator new
95         (aCapacity * sizeof(value_type)));
96     size_t lCount = count();
```

```
97
98     for (size_t i = 0; i < lCount; i++)
99     {
100         new (&lNewElements[i]) value_type(fElements[i + fHead]);
101         fElements[i + fHead].~value_type();
102     }
103
104     ::operator delete(fElements);
105
106     fElements = lNewElements;
107     fHead = 0;
108     fTail = lCount;
109     fCapacity = aCapacity;
110 }
111
112 template<typename T, typename P>
113 void PriorityQueue<T, P>::ensure_capacity()
114 {
115     if (fTail == fCapacity)
116     {
117         resize(fCapacity * 2);
118     }
119 }
120
121 template<typename T, typename P>
122 void PriorityQueue<T, P>::adjust_capacity()
123 {
124     if (fCapacity > 1 && count() * 4 <= fCapacity)
125     {
126         size_t lNewCapacity = fCapacity / 2;
127         if (lNewCapacity < 1) lNewCapacity = 1;
128         resize(lNewCapacity);
129     }
130 }
131
132 template<typename T, typename P>
133 void PriorityQueue<T, P>::push(const T& aValue, const P& aPriority) ↗
134     noexcept
135 {
136     ensure_capacity();
137     new (&fElements[fTail++]) value_type(aPriority, aValue);
138     std::stable_sort(&fElements[fHead], &fElements[fTail]);
139 }
140
141 template<typename T, typename P>
142 void PriorityQueue<T, P>::pop() noexcept
143 {
144     assert(count() > 0);
145     fElements[fHead].~value_type();
```

```
145     fHead++;  
146     adjust_capacity();  
147 }
```